VERA-SE: Verification-Guided Autonomous Software Repair with Risk-Aware Patch Selection

Mridula Prabhakar

Abstract

Large language models can produce plausible software patches from natural-language bug descriptions and localised source-code context. However, it is still a challenge to assess the correctness of a generated patch. Passing the existing tests may not be enough, because test suites can be incomplete and patches that are syntactically correct can introduce behavioural regressions or unrelated changes. In this paper, we propose VERA-SE, a verification-guided autonomous software repair framework that combines LLM-based candidate generation with structural analysis, behavioural preservation checks, change-risk estimation, and verification-aware patch selection.

VERA-SE does not just pick the first patch that passes the visible tests, but instead considers many candidate repairs. Before a decision on acceptance can be made, the candidates are evaluated for syntactic validity, target modification, repair-scope preservation, structural consistency, behavioural properties where available and change risk. We test the approach in two settings. First, we compare visible-test-only acceptance, visible-plus-hidden-test acceptance and VERA-SE with a controlled benchmark of six repair tasks and 240 candidate instances . Acceptance based on visible tests alone led to a false acceptance rate of 74.7%, which dropped to 27.6% when hidden tests were included. VERA-SE's false-acceptance rate was 0% and the false-rejection rate was one out of 153 oracle-correct candidates. Ablation analysis shows that the reduction is mostly due to behavioural preservation checks, with structural checks providing limited additional false-acceptance benefit, and adding one false rejection. Because the controlled probes and the held-out oracle were written from the same task specification, and overlap substantially in content, the controlled result should be read as a demonstration of the mechanism rather than as an estimate of the improvement obtainable when probes are not available.

Second, we perform a real-world pilot on ten localised defects of the Black project in BugsInPy. The system generated 28 valid candidate instances over 40 requested repair generations; only one satisfied the held-out correctness oracle. In this case, the visible-test baseline and VERA-SE each produced one false acceptance while VERA-SE also rejected the sole oracle-correct candidate. Results show that verification-guided repair can improve reliability when verification signals are representative of behavioural constraints, but demonstrate a transfer gap when generic structural and risk signals are applied to real-world defects. This motivates more robust automated semantic verification for reliable LLM-based program repair.

Keywords—automated program repair, large language models, patch validation, behaviour verification, patch overfitting, risk-aware selection.

1. Introduction

Large language models are capable of generating, modifying, and fixing source code. Code-oriented models are capable of generating syntactically plausible patches that seem to fit within the surrounding program context, given a bug report, failing test or localised code fragment. These capabilities have inspired autonomous software repair systems that take a faulty region as input, propose modifications, and apply a patch with limited human intervention.

However, generating a plausible patch is not equal to generating a correct patch. A candidate can pass tests that are visible to the repair system, while violating untested behaviour, changing unrelated functionality, or implementing an overly broad approximation of the intended fix. This distinction is important for LLM-based repair, as a fluent code suggestion is not a semantic guarantee. Autonomous repair therefore requires a generation mechanism and a principled way to evaluate which candidates should be trusted.

Test-suite driven automated program repair often uses the successful execution of available tests as the primary validation criterion [1], [6]. Tests provide executable evidence but rarely provide complete specifications. So a patch might be plausible on the test cases we saw, but wrong elsewhere. In the literature, this is known as patch overfitting [6], [7]. An LLM might generate multiple different repairs for a single failure, some of which are

DOI: 10.5281/zenodo.23440127

that may change too much code, remove logic, or only handle the triggering example. Visible-test success alone may not distinguish these candidates.

Visible-test alone may not distinguish these candidates. This work studies whether verification-guided candidate selection can help to reduce false acceptance. We present VERA-SE, a verification-guided autonomous software repair framework with risk-aware patch selection. The generation and validation are two separate stages: a localised set of candidates is produced by a local code-oriented LLM, and a specific verifier checks the syntax, target modification, repair-scope preservation, structural changes, behavioural checks when available, and change risk. This evidence is used by the framework in its acceptance and selection decisions.

The basic idea is that the plausibility of a patch should not be judged solely by tests that are visible at repair time. Independent evidence on program behaviour, structural locality and unnecessary modification could be useful. At the same time, these signals are not interchangeable: a small or structurally localised patch is not necessarily semantically correct. We first test this hypothesis on a controlled benchmark with 6 repair tasks and 240 candidate instances. We compare three strategies: visible only, visible and hidden, and VERA-SE. Visible-only acceptance accepts all 153 correct candidates, and 65 incorrect candidates, for a false-acceptance rate of 74.7% out of 153 correct and 87 incorrect candidates. Visible-plus-hidden testing accepts all correct candidates and 24 incorrect candidates, lowering false acceptance to 27.6% . VERA-SE: 152 correct candidates accepted 0 incorrect candidates accepted False acceptance = 0 % False rejection = ∼=0.7 % The improvement in false acceptance in ablation analysis is mainly due to behavioural preservation, and the structural checking does not bring any similar improvement.

Controlled tasks allow specification of relevant behavioural properties, which may overstate transfer to realistic repositories. We therefore run a pilot on ten localised BugsInPy defects from the Black project. The oracle used for evaluation is withheld and is not shown to the generator or VERA-SE during candidate selection. We asked for forty candidate generations and got 28 syntactically valid candidate instances that represented 27 unique patch contents. Only one candidate passed the held-out oracle. The visible-test baseline accepted the candidate and one wrong candidate, while VERA-SE accepted the wrong candidate and rejected the right one.

The contrast is an important result. The controlled experiment demonstrates that verification can substantially decrease false acceptance if the checks are consistent with the intended behaviour. The real-world pilot suggests that generic structural and change-risk evidence may not be enough to discriminate semantically correct repairs. The challenge therefore is not only to add filters, but to get more informative properties about the actual program behaviour.

The contributions of this paper are fourfold. First, it proposes a modular generate-and-verify pipeline for multiple LLM-generated repair candidates. Second, it compares three acceptance strategies on a controlled benchmark, and analyses the contribution of individual verification signals. Third, it tests the prototype on held-out tests on ten localised defects from one real-world project, recording both generation failure and incorrect verifier decisions. Fourth, it describes a transfer gap between task-specific behavioural verification and generic real-world patch-risk signals, motivating stronger automated semantic properties. The rest of the paper covers related work, the VERA-SE architecture and verification model, the controlled and real-world methodologies, the results and ablation findings, the implications of the observed transfer gap, threats to validity, and conclusions.

2. Related Work

2.1 Automated Program Repair

Automated Program Repair (APR) is concerned with reducing the effort required to fix software defects by automatically generating modifications to faulty programs. Proposed candidate changes are evaluated with tests, program analysis, search, historical fix patterns, and semantic constraints.

GenProg is a technique introduced by Le Goues et al [1]. that uses genetic programming based on existing tests to evolve variants of a program to fix defects without requiring full formal specifications. This shows the potential of automated repair, but the test-based validation depends on the quality and coverage of the test suite.

Bader et al [2]. proposed Getafix [2], which learns the repair patterns from human-written fixes and ranks the suggestions using code-change context. Angelix is developed by Mechtaev et al[3] and synthesises multi-line patches based on symbolic analysis and semantic constraints. These approaches demonstrate different ways to guide the search space of repairs using structural or semantic evidence.

VERA-SE adopts the broader generate-and-validate paradigm, but focuses on acceptance and selection after the

generation of multiple LLM-generated candidates. Its central question is whether acceptance of incorrect patches decreases more than visible-test success with more evidence.

2.2 LLM-Based Software Repair

Large language models enable us to fix programs from natural-language instructions, failing tests, and repository context. Language models can suggest a wide variety of source-code modifications, unlike systems constrained to a set of pre-determined transformations.

Jimenez et al. [4] proposed SWE-bench to evaluate language models on real GitHub issues, which often require reasoning across multiple functions, files or execution conditions. Yang et al. [5] propose SWE-agent, an agent-computer interface for navigation in repositories, editing code, executing commands and testing. These efforts demonstrate the role of interaction with the environment in autonomous software engineering.

However, code generation and successful execution of test cases do not always indicate that the code is semantically correct. VERA-SE separates generation and verification: a local code-based model proposes local repairs and a separate evaluation pipeline determines whether they should be accepted. The question is not only whether the model can propose a plausible fix, but whether verification can distinguish correct candidates from incorrect ones.

2.3 Patch Overfitting and Correctness Assessment

Patch overfitting is the case that a repair passes the tests that were used to generate it, but it does not satisfy behaviour not covered by tests. Smith et al. [6] tested automatically generated repairs with independent tests and stressed the limitations of using the same suite for both repair and correctness evaluation.

Ye et al. [7] studied automated patch assessment at scale, using random testing with a ground-truth implementation as additional behavioural evidence to detect overfitting. Such work emphasises the necessity of independent verification, as well as the difficulty of automatic assessment of correctness.

These results motivate the distinction between test information available at generation and held-out correctness oracle in our real-world pilot. Passing the finite oracle is interpreted as an operational correctness label for the behaviours tested, not a formal proof that a patch preserves all semantics of a program.

2.4 Verification-Guided and Risk-Aware Repair

Semantics-based methods, such as Angelix [3], use behavioural constraints to narrow down the space of possible repairs. Other independent patch-assessment methods such as ground-truth-based testing [7] can uncover failures that the repair-time tests do not. These contributions support the inclusion of evidence other than visible-test results in repair assessment.

Vera-se evaluates syntactic correctness, target modification, repair-scope preservation, structural changes, available behavioural properties and change-risk estimates. Behavioural checks are associated with execution properties while structural and risk signals describe the amount and locality of source changes.

These signals should not be confused with semantic correctness. Even a small patch can still perform the wrong behaviour, whereas a correct patch may require a larger change. The controlled ablation study and real-world pilot are designed to reveal these differences rather than assuming that all verification components contribute equally.

2.5 Positioning of VERA-SE

Existing work on automatic program repair includes search-based approaches, learned fix patterns, symbolic synthesis, language-model-based repository repair, and independent patch evaluation. VERA-SE investigates the impact of verification and change-risk signals on acceptance and selection among multiple LLM-generated repairs.

Its controlled benchmark measures the usefulness of behaviorally-informative checks, while the held out real-world pilot tests transfer to localised defects in an existing repository. This set of experiments collectively motivates the creation of more powerful automated approaches for deriving semantic verification properties, rather than generic structural risk signals only.

3. VERA-SE Approach and System Architecture

3.1 System Overview

VERA-SE is a modular framework for generation and evaluation of candidate software repairs. It decouples patch generation from patch verification so that a model proposed change is not automatically accepted just because it is syntactically plausible or passes an available test. A repair task gives faulty source code, a target repair location and validation information. The generator provides a set of candidates. The evaluation pipeline gathers independent evidence for each candidate and records an acceptance decision. The controlled-benchmark pipeline includes candidate generation, test execution, structural analysis, task-specific behavioural verification, change-risk estimation, and patch selection.

If multiple candidates meet all acceptance criteria, the selector returns the accepted candidate with the highest verification score. If there is no patch satisfying the conditions, the system does not select a patch. The distinction between generation success and test success, verification acceptance and final selection.

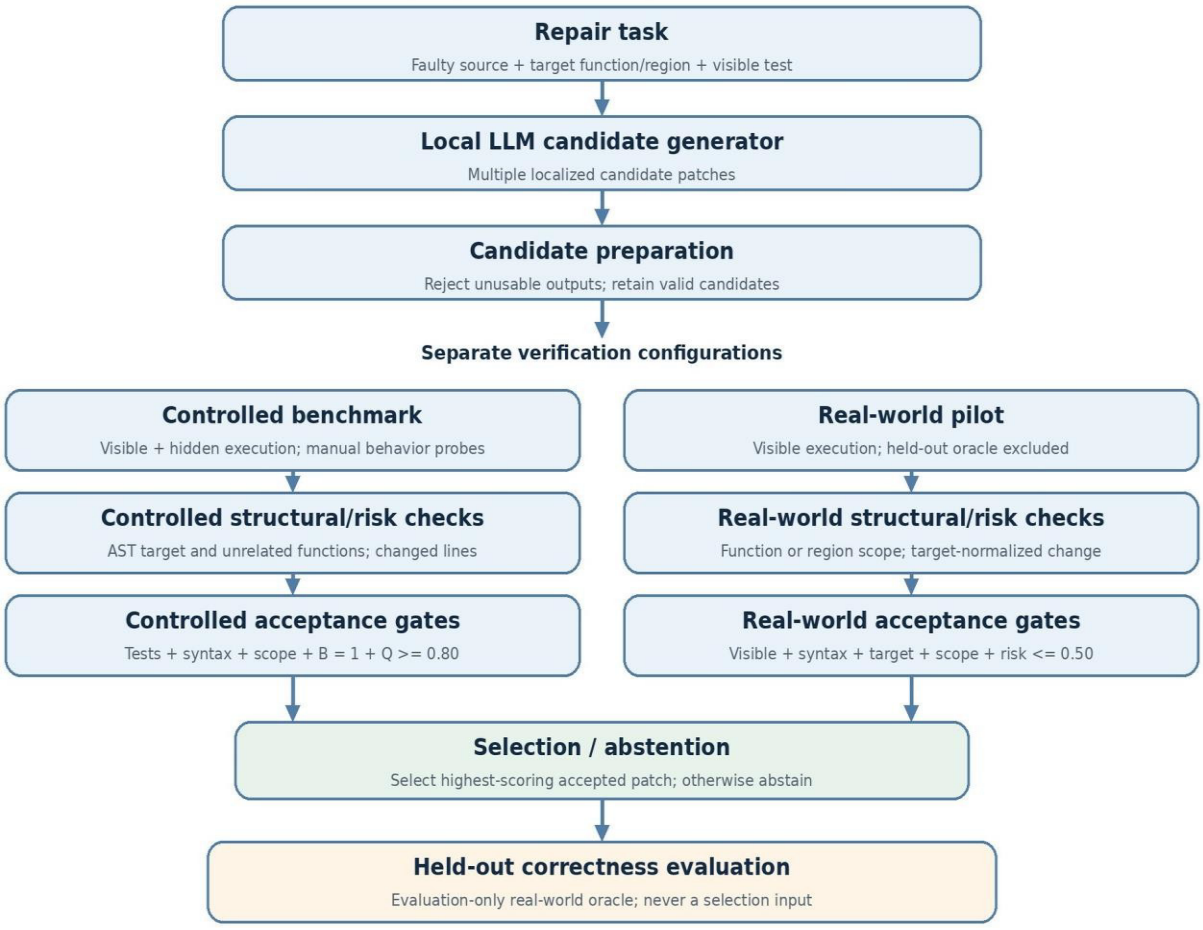


Figure 1. VERA-SE architecture, distinguishing the controlled and real-world verifier configurations. The held-out real-world oracle is evaluation-only.

3.2 Candidate Generation

The controlled-benchmark generator communicates with a locally hosted code-oriented language model and asks for structured candidate information: candidate identifier, target file path, full patched Python source, and a short explanation. It skips an empty patch and rejects a response with a file path that doesn't match the source file requested. CandidatePatch objects that correspond to valid answers are passed into the evaluation pipeline. Candidate generation is thus separate from candidate acceptance; a patch returned must still satisfy the verifier.

3.3 Structural Verification

The structural analyser parses the original and patched code using the Python abstract syntax tree (AST) to extract function definitions, including asynchronous functions. It compares the corresponding source spans for each function name to find the functions that have been modified, added or deleted. Suppose the original and patched function names are F_o and F_p which denote the sets of original and patched function names, and let $S_o(f)$ and $S_p(f)$ denote the extracted source for function f. The changed-function set is:

$$\Delta F = \{ f \in F_o \cup F_p : S_o(f) \neq S_p(f) \}.$$

For target function f_t, unrelated-function changes are $U = \Delta F \setminus \{f_t\}$. The controlled verifier requires a syntactically valid patch, a modification to the designated target function, and no unrelated-function changes. These constraints address repair locality; they do not independently prove semantic correctness.

3.4 Behavioral Verification

Task-specific executable probes test properties other than visible test cases in the controlled benchmark. Probes include numeric results, string normalisation, input-container preservation, return types, and repeatability as appropriate to the task. The analyser writes the patched source and probe program to a temporary directory, runs the probe with a time limit, parses the JSON Boolean outcomes, and counts successful checks for each candidate. The behavioral score is:

$$B(p) = N_{passed} / N_{total}, \text{ for } N_{total} > 0.$$

If probe timeout, unsuccessful process exit, or invalid JSON, score is zero. The controlled acceptance rule requires that all the behavioural checks specified pass, $B(p) = 1$. For the six controlled tasks, the probes are manually provided and should not be described as automatically derived semantic specifications for arbitrary real-world repositories. The probes and the held-out oracle were written from the same task specification and overlap in content: the label-formatting probe reproduces every oracle example, the numeric-averaging probe five of the oracle's eight, and the dictionary-access probe one oracle test exactly with two more over different data.

3.5 Change-Risk Estimation

The verifier computes the number of added and removed source lines from a unified diff. Let L_a and L_r be the number of lines added and removed and let L_o be the original source line count. Its normalized change-risk proxy is:

$$R(p) = \min(1, (L_a + L_r) / \max(L_o, 1)).$$

It also penalises modified functions outside of the target:

$$P_u(p) = \min(0.15 \times |U|, 0.30).$$

These quantities measure the size and locality of the modification; R(p) is not a calibrated probability of semantic failure. A small patch can be wrong and a correct patch can legitimately have a large change.

3.6 Verification Score and Acceptance

The controlled verifier combines visible-test pass ratio T_v, hidden-test pass ratio T_h, behavioral score B, syntax-validity indicator S, normalized change risk R, and unrelated-function penalty P_u. The score is clamped to be in the interval [0, 1]:

$$Q(p) = \text{clamp}_{[0,1]}(0.25T_v + 0.30T_h + 0.25B + 0.10S + 0.10(1 - R) - P_u).$$

The controlled implementation also requires successful exit codes for visible and hidden tests, syntax validity, target-function modification, no unrelated-function changes, complete behavioral-probe success, and $Q(p) \geq 0.80$:

$$A(p) = Vpass \wedge Hpass \wedge S \wedge (f_p \in \Delta F) \wedge (|U| = 0) \wedge (B = 1) \wedge (Q \geq 0.80).$$

This is a conjunction of hard constraints, not a threshold on score. In this controlled configuration the hidden-test input is part of the validation evidence of the verifier. The separate held-out correctness oracle used to label real-world candidates must not be confused with a verifier-accessible hidden-test.

3.7 Risk-Aware Patch Selection

The selector forms the accepted set $P_a = \{p \in P : A(p) = 1\}$. If P_a is nonempty, it returns the highest scored accepted patch; else it returns no patch:

$$p^* = \arg\max_{p \in P_a} Q(p), \text{ if } P_a \neq \varnothing \text{ ; otherwise abstain.}$$

Consequently, the highest scoring generated candidate may not be eligible to be selected. Only candidates passing the mandatory acceptance checks are ranked by their score.

3.8 Real-World Repair Configuration

The real-world pilot supports both function level and contiguous region repairs. The dedicated verifier uses a visible triggering test, Python syntax validity, change of the designated target, preservation of authorised repair scope, and a source-diff change-risk heuristic. The real-world verifier is not provided with the manual behavioural probes used in the controlled benchmark. The verifier only evaluates the held-out oracle after he has decided if he is going to accept a candidate.

For function-level repairs, ASTAnalyzer compares representations of function sources, requires the target function to change, and treats changes to other named functions as scope violations. Region-level repairs require that replacements and deletions occur within the supplied original-code interval, with insertions at the ends. Both configurations measure the original target size L_s in-line.

$$Rreal(p) = min(1, (La + Lr) / max(Ls, 1)).$$

The ranking score is based on the indicators V, S, P, and T for visible-test success, syntax validity, scope preservation, and target modification, respectively:

$$Qreal(p) = clamp[0,1](0.45V + 0.15S + 0.15P + 0.10T + 0.15(1 - Rreal)).$$

Acceptance is determined by explicit gates, and not by than a minimum ranking score:

$$Areal(p) = V \wedge S \wedge P \wedge T \wedge (Rreal(p) \leq 0.50).$$

The score does not override a failed gate. It ranks or records evidence for candidate patches. For example, a candidate might pass the visible test and satisfy the held-out oracle, but still be rejected because they exceed the risk threshold. This distinction is key in the case analysis of Black-9 in section 5.5.

4. Experimental Methodology

4.1 Research Questions

RQ1: How does verification-guided acceptance compare with visible-test-only and visible-plus-hidden-test acceptance on the controlled tasks? RQ2: What components cause changes in false acceptance and false rejection? RQ3: How does the prototype perform on localised real-world defects in the absence of task-specific behavioural probes?

4.2 Controlled Benchmark

The controlled benchmark contains 6 Python tasks: percentage calculation, string normalisation, list access, dictionary access, label formatting, and numeric averaging. For each task, we produced 10 repeated generations, and each generation produced 4 candidate instances, totalling 240 instances (6 x 10 x 4). The evaluation labels contain 153 correct and 87 incorrect candidate instances. Repeated generations in a task are not independent software bugs.

We compare (i) visible-test-only acceptance, (ii) visible-plus-hidden-test acceptance, and (iii) the complete VERA-SE controlled verifier, which incorporates test outcomes, AST-based scope checks, task-specific behavioral probes, and change-risk information. In the complete controlled verifier, the additional hidden tests and behavioral probes are available during the acceptance decision; this is an evaluation of the complete configuration and not of risk scoring in isolation.

4.3 Real-World Pilot and Scope

The pilot consists of ten localised defects from the Black Python formatter as indexed by BugsInPy [8], namely Black-1, Black-3, Black-4, Black-7, Black-8, Black-9, Black-10, Black-13, Black-14 and Black-16. Inclusion required a localised function or contiguous region, a reproducible trigger test, and an executable dependency/fixture environment. The pilot did not address changes that spanned multiple files or the architecture. Therefore, the results characterise these Black defects, rather than BugsInPy.

Each task had four requested candidate-generation slots with a local Qwen2.5-Coder 7B model on Ollama. Invalid generations were unsuccessful slots and were not replaced again until four valid patches were acquired. The frozen evaluation is based on run IDs 20260904_164914 and 20260904_171236. (Excludes a modified and interrupted subsequent generator run.).

4.4 Independent Correctness Assessment

A separate held-out oracle was performed after candidate generation and VERA-SE acceptance decisions. A candidate was assigned an oracle-positive label if and only if the oracle process was successful, at least one test passed and no test failed. Developer-fix information was inspected privately when localising benchmarks and building oracles but was not provided to the model prompt or the verifier. These finite held-out tests are a proxy for correctness of behaviours tested, not a formal proof of semantic equivalence.

4.5 Metrics and Units of Analysis

A positive decision indicates the candidate patch is accepted. A positive reference label indicates it passed the held-out correctness criterion. Valid candidate instances are used to calculate TP, FP, TN and FN. We report accuracy, precision, recall, false acceptance rate (FAR) and false rejection rate (FRR).

$$Accuracy = (TP + TN) / (TP + FP + TN + FN); Precision = TP / (TP + FP).$$

$$Recall = TP / (TP + FN); FAR = FP / (FP + TN); FRR = FN / (TP + FN).$$

We separately report requested generations, valid candidate instances, unique patch contents, and tasks containing at least one valid candidate. Candidate-level metrics must not be interpreted as independent-bug success rates.

5. Experimental Results

5.1 Controlled Benchmark

Table 1. Controlled benchmark: candidate-level acceptance (240 instances).

Metric	Visible only	Visible + hidden	VERA-SE
True positives	153	153	152
False positives	65	24	0
True negatives	22	63	87
False negatives	0	0	1
Accuracy	72.9%	90.0%	99.6%
Precision	70.2%	86.4%	100.0%
Recall	100.0%	100.0%	99.3%
FAR	74.7%	27.6%	0.0%
FRR	0.0%	0.0%	0.7%

Visible testing accepted all 153 labeled-correct candidate instances, but also accepted 65 of the 87 incorrect instances. Additional hidden tests brought down the false acceptance to 24 wrong cases. The full VERA-SE controlled setting disallowed all 87 incorrect cases and disallowed one correct case. These results support the utility of behaviourally informative verification within this small controlled setting.

5.2 Controlled Ablation Findings

To evaluate the impact of single verification signals, we compare five acceptance configurations on the same 240 controlled candidate instances. The accuracy, precision, recall, false-acceptance rate (FAR) and false-rejection rate (FRR) are summarised in Table 2.

Table 2. Controlled benchmark ablation results (240 candidate instances).

Configuration	Accuracy	Precision	Recall	FAR	FRR
Visible tests only	72.9%	70.2%	100.0%	74.7%	0.0%
Visible + hidden	90.0%	86.4%	100.0%	27.6%	0.0%
Visible + hidden + behavior	100.0%	100.0%	100.0%	0.0%	0.0%
Visible + hidden + structure	89.6%	86.4%	99.3%	27.6%	0.7%
Full VERA-SE	99.6%	100.0%	99.3%	0.0%	0.7%

Visible-only validation accepted 65 of 87 wrong candidates incorrectly. With the addition of hidden tests, the false acceptance rate dropped to 24, or 27.6%.

In this controlled dataset, the configuration of visible tests, hidden tests and behavioural preservation checks accepted all 153 correct candidates and rejected all 87 incorrect candidates. Thus, it had zero observed false acceptance and zero observed false rejection.

Structural verification alone without behavioural preservation could not reduce false acceptance further compared to visible-plus-hidden validation. It rejected one genuine claimant, giving a FRR of 0.7 %. The full VERA-SE configuration removed all 24 remaining false acceptances, but kept the one false rejection from structural verification.

These results suggest that the additional testing does not explain the reduction in false acceptance observed after a task-specific behavioural preservation check. Evidence of the scope of repair was provided by structural constraints, but on their own they did not improve the discrimination of incorrect patches in this benchmark. The results are about repeated candidate instances from six controlled tasks and not independent real-world defects.

5.3 Real-World Generation Outcomes

Table 3. Real-world pilot generation summary.

Measure	Observed value
Localized Black defects	10
Requested generations	40
Valid candidate instances	28
Unique full patched-source hashes	27
Valid-generation rate	70.0%
Tasks with ≥1 valid candidate	8/10
Oracle-positive candidate instances	1

Black-13 and Black-16 failed to produce any valid candidates in any of the four generations requested. Black-1 and Black-8 created three, Black-14 created two, and the other five tasks created four each. The determinant of the requested generation includes invalid generations. The following classification metrics are computed on the 28 valid candidate instances.

5.4 Real-World Verification Outcomes

Table 4. Real-world pilot: candidate-level classification (28 valid instances).

Metric	Visible baseline	VERA-SE
TP	1	0
FP	1	1
TN	26	26
FN	0	1
Accuracy	96.4%	92.9%
Precision	50.0%	0.0%
Recall	100.0%	0.0%
FAR	3.7%	3.7%
FRR	0.0%	100.0%

The visible baseline accepted two candidates, one oracle-positive and one oracle-negative. VERA-SE rejected the only oracle-positive candidate and accepted one oracle-negative candidate. Thus, in this pilot its false acceptance rate was seen to be equal to and not better than the visible baseline. Recall and FRR are defined by a single positive instance each and are not stable population estimates.

5.5 Black-9: A Correct Patch Rejected by the Risk Gate

Black-9 contains the only oracle-positive patch in the frozen real-world pilot. Candidate 2 passed the visible test and was accepted by VERA-SE with score 0.945 and normalised risk 0.364, but passed only one of three held-out oracle tests . Candidate 4 was rated 0.905 on the visible test and all three oracle tests, but its risk 0.636 was higher than the mandatory maximum of 0.50. The gate rejected it, although it had a higher held-out correctness label.

Table 5. Black-9 case study.

Candidate	Visible	Risk	VERA decision	Oracle
2	Pass	0.364	Accept	Incorrect (1/3 passed)
4	Pass	0.636	Reject	Correct under oracle (3/3 passed)

This observed error highlights the difference between modification size and semantic correctness: a smaller modification might be incomplete, and a larger localised fix might satisfy the tested intended behaviour. The case does not show oracle leakage, since oracle is used after the decision.

6. Discussion

The two evaluation settings give different results. Controlled task-specific behavioural probes were strongly discriminative and eliminated false acceptance on the 240-instance benchmark. That reduction was not replicated in real-world setup, visible tests, and general syntax, scope, target-change, and risk signals. These are different configurations of the verifier and the difference cannot be explained by synthetic vs. real-world task difficulty alone.

The Black-9 case reveals a particular failure mode. The 0.50 change-risk gate did not select the sole oracle-positive candidate, but did select a smaller incomplete candidate. Thus the heuristic is useful as a bound on the size of modification, but it cannot be taken as a probability of correctness or a substitute for semantic evidence.

The effectiveness of the generation is also a limitation of the pilot. 12 out of 40 requested generations returned no valid candidate instance at all, and only one out of 28 valid candidates passed the oracle. A selector cannot recover a correct repair when its candidate pool has none. In future development we should separate improvements in candidate generation from improvements in verification and not tune the reported frozen evaluation after seeing the held out labels.

The results are promising for the automatic derivation of behavioural properties from code context, specifications, tests, execution traces or invariants. This is an indicative direction rather than a functionality exemplified by the current real-world prototype.

7. Threats to Validity

7.1 Construct and Oracle Validity

Finite visible, hidden, behavior-probe and oracle tests are approximations of intended semantics, not proofs of total correctness. Controlled probes are hand-specified and overlap substantially with the held-out oracle, in one case completely. Where that overlap is high a candidate is unlikely to pass the probe and fail the oracle, so the reported 0% controlled false-acceptance rate is partly a consequence of probe design rather than an independent measurement. The real-world pilot, where no such probes were supplied, is the more informative estimate. In the real world, the oracle has a different role: it gives labels after the verifier made its decision.

7.2 Internal Validity

The controlled and real-world experiments use different acceptance rules and different behavioral information. Thus their results cannot isolate dataset complexity from verifier configuration. Developer patches informed real-world benchmark preparation but were not included in generation/selection inputs. The real-world risk threshold is a fixed heuristic, not a calibrated probability.

7.3 External Validity

Six controlled tasks and ten localised defects from one real-world project are insufficient to make general claims over languages, repositories, or all of BugsInPy. The 240 controlled observations are repeated candidate instances for six tasks, not independent bugs.

7.4 Statistical Validity

Out of the 28 valid real-world candidate instances, only one is oracle-positive. The precision and recall values, and in particular VERA-SE's 0% recall and 100% FRR, are highly sensitive to this single observation. Candidate events from the same defect are correlated and no statistically powered generalisation is claimed.

7.5 Reproducibility

The frozen artifacts include task and candidate CSVs, requested-versus-valid counts, patched code, diffs, and source hashes. The authoritative pilot uses runs 20260904_164914 and 20260904_171236; a later generator modification and interrupted run are excluded from the analysis. SHA-256 hashes have been recorded for the authoritative result files to support artifact-integrity checks. Git history was initialized after the experiments, so the current repository snapshot does not establish the exact historical generator revision. Accordingly, the frozen artifacts are treated as the authoritative experimental record, and end-to-end reproduction should not be claimed until the original generation configuration is reconstructed or independently preserved.

8. Conclusion

VERA-SE seperates LLM-generated patch proposals from explicit verification-driven acceptance and risk-aware selection. In a controlled benchmark with six tasks, task-specific behavioural verification achieved a zero false acceptance rate on 87 incorrect candidate instances, but at the cost of one false rejection on 153 correct instances. For the ten-defect Black pilot, the system produced 28 valid candidates from 40 requested generations; only one met the held-out oracle. The real-world verifier incorrectly accepted one patch and incorrectly rejected the single oracle-positive patch, matching the false-acceptance count of the visible baseline rather than improving it.

These results distinguish the demonstrated potential of meaningful behavioral verification from the limitations of generic structural and change-risk filtering. More informative automatically derived semantic properties and broader, reproducible multi-project evaluations are needed before stronger claims about real-world repair reliability can be made.

References

[1] C. Le Goues, T. V. Nguyen, S. Forrest, and W. Weimer, "GenProg: A Generic Method for Automatic Software Repair," IEEE Transactions on Software Engineering, vol. 38, no. 1, pp. 54–72, 2012. doi: 10.1109/TSE.2011.104.

[2] J. Bader, A. Scott, M. Pradel, and S. Chandra, “Getafix: Learning to Fix Bugs Automatically,” Proc. ACM Program. Lang., vol. 3, OOPSLA, Art. no. 159, pp. 1–27, 2019. doi: 10.1145/3360585.

[3] S. Mechtaev, J. Yi, and A. Roychoudhury, “Angelix: Scalable Multiline Program Patch Synthesis via Symbolic Analysis,” in Proceedings of the 38th International Conference on Software Engineering, 2016, pp. 691–701. doi: 10.1145/2884781.2884807.

[4] C. E. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” in Proc. Int. Conf. Learn. Representations (ICLR), 2024.

[5] J. Yang et al., “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” in Adv. Neural Inf. Process. Syst., vol. 37, 2024. doi: 10.52202/079017-1601.

[6] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, “Is the Cure Worse Than the Disease? Overfitting in Automated Program Repair,” in Proceedings of ESEC/FSE, 2015, pp. 532–543. doi: 10.1145/2786805.2786825.

[7] H. Ye, M. Martinez, and M. Monperrus, “Automated Patch Assessment for Program Repair at Scale,” Empirical Software Engineering, vol. 26, article 20, 2021. doi: 10.1007/s10664-020-09920-w.

[8] R. Widyasari et al., “BugsInPy: A Database of Existing Bugs in Python Programs to Enable Controlled Testing and Debugging Studies,” in Proc. 28th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE), 2020, pp. 1556–1560. doi: 10.1145/3368089.3417943.